

myownX 社交网络平台

系统设计说明书

2026 年 7 月 8 日

1 系统概述

myownX 是一个面向课堂演示场景的 Web 社交网络平台。系统支持用户注册登录、个人资料管理、文字动态和图片动态发布、信息流浏览、关注、点赞、评论、一对一私信、通知、发现页推荐和基础隐私设置等功能。

系统采用本地运行方式，用户通过电脑浏览器或手机浏览器访问系统。系统重点保证核心社交流程能够正常演示，暂不考虑高并发、分布式部署、复杂推荐算法和商用级安全防护。

2 系统体系架构

系统采用分层架构设计，主要包括用户终端层、前端展示层、Flask 控制层、业务逻辑层、数据访问层、数据库和文件存储层。

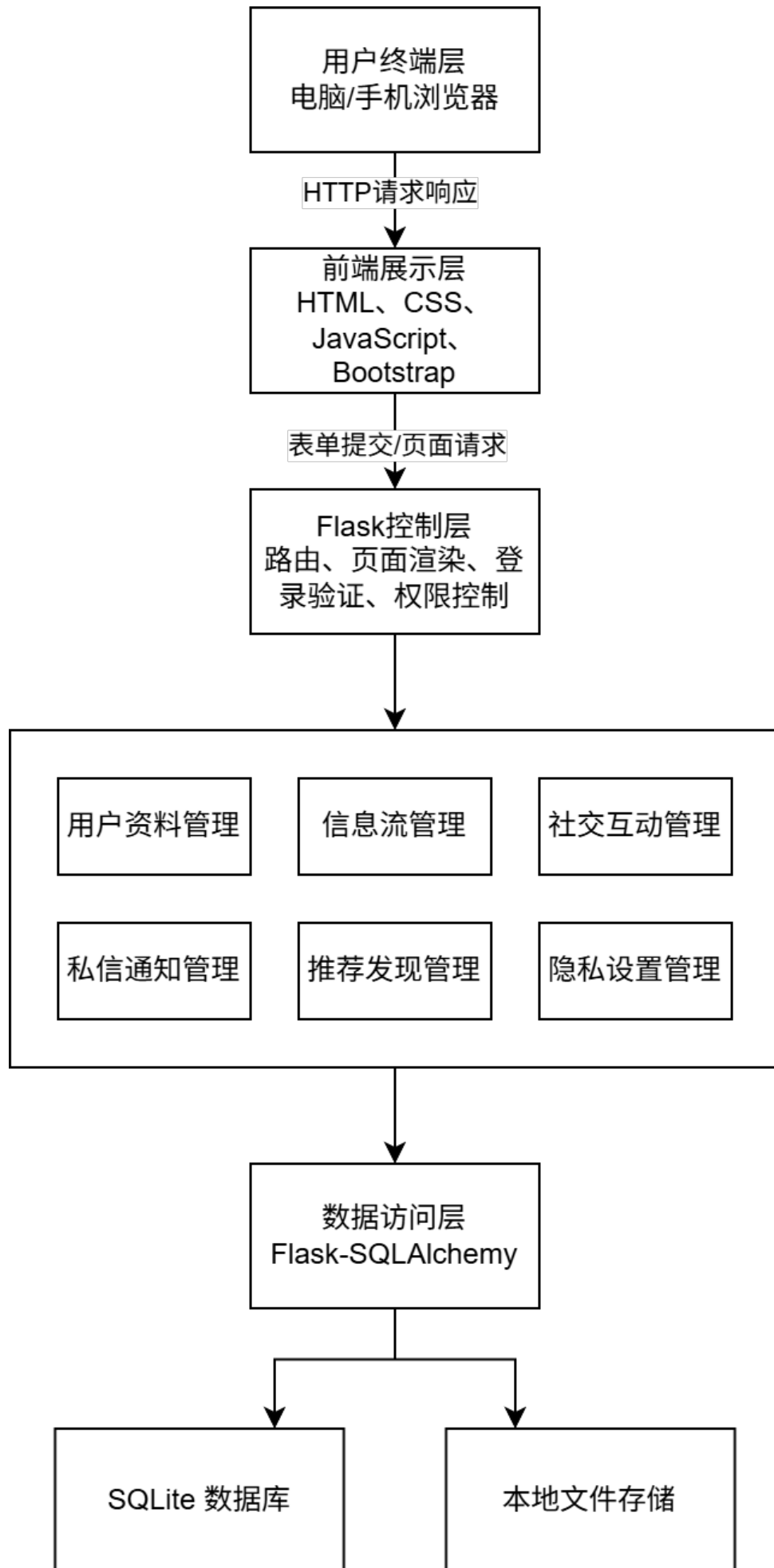


图 1: myownX 系统体系架构图

用户通过电脑或手机浏览器访问系统页面。前端展示层使用 HTML、CSS、JavaScript 和 Bootstrap 5 实现页面展示和响应式布局。Flask 控制层负责处理路由、表单请求、登录状态验证和权限判断。业务逻辑层负责用户资料、动态信息流、社交互动、私信通知、发现推荐和隐私设置等功能处理。系统通过 Flask-SQLAlchemy 访问 SQLite 数据库, 用户头像和动态图片保存于本地文件目录中。

3 系统功能结构设计

系统按照业务功能划分为用户与资料管理、动态与信息流管理、社交互动管理、私信与通知管理、发现与推荐管理、隐私设置管理六个主要模块。

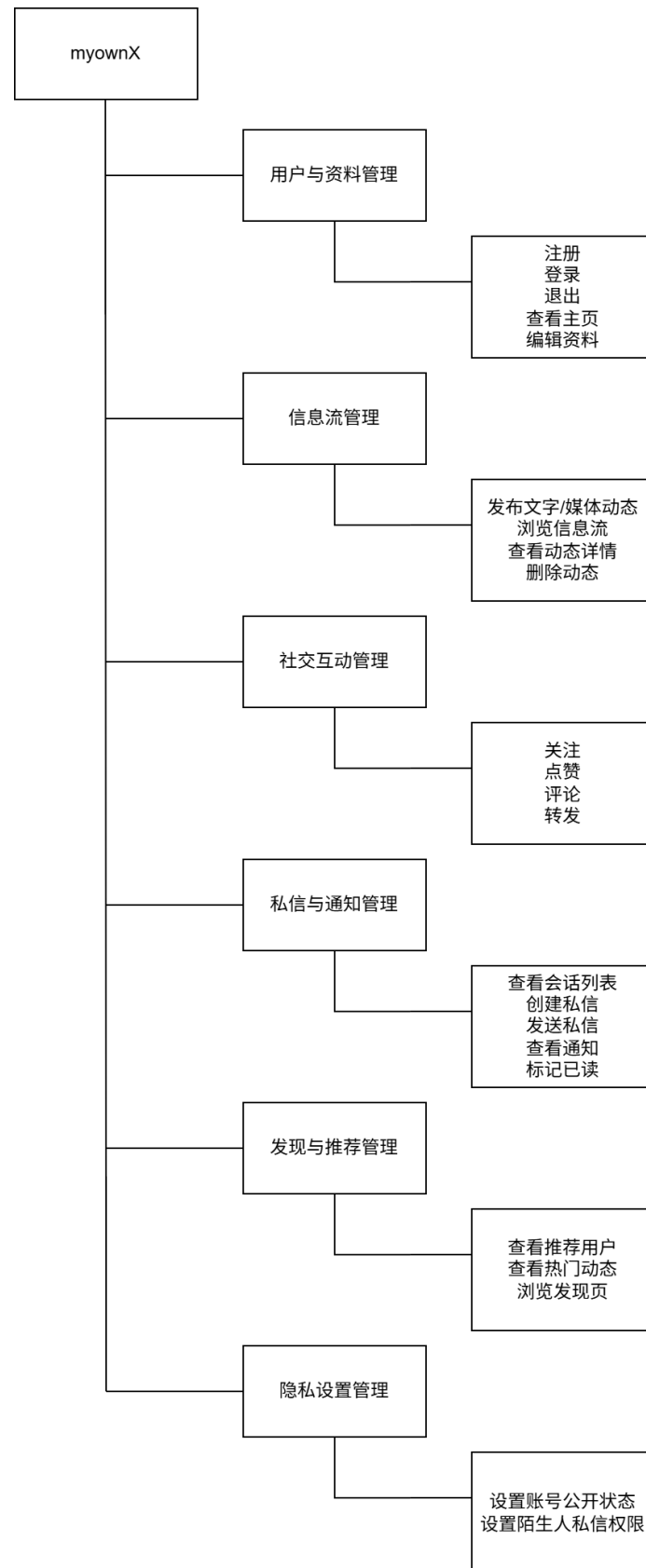


图 2: myownX 系统功能结构图

各模块说明如下：

- 用户与资料管理：负责用户注册、登录、退出、个人主页查看和资料编辑；
- 动态与信息流管理：负责文字动态、图片动态发布，以及信息流展示；
- 社交互动管理：负责关注、取消关注、点赞、取消点赞和评论；
- 私信与通知管理：负责一对一私信、会话列表、消息发送和通知展示；
- 发现与推荐管理：负责展示推荐用户和热门公开动态；
- 隐私设置管理：负责账号公开状态和陌生人私信权限设置。

4 系统用例时序设计

本系统选取“发布动态”作为核心用例进行时序设计。该用例覆盖了用户操作、页面提交、后端验证、文件保存和数据库写入等主要过程。

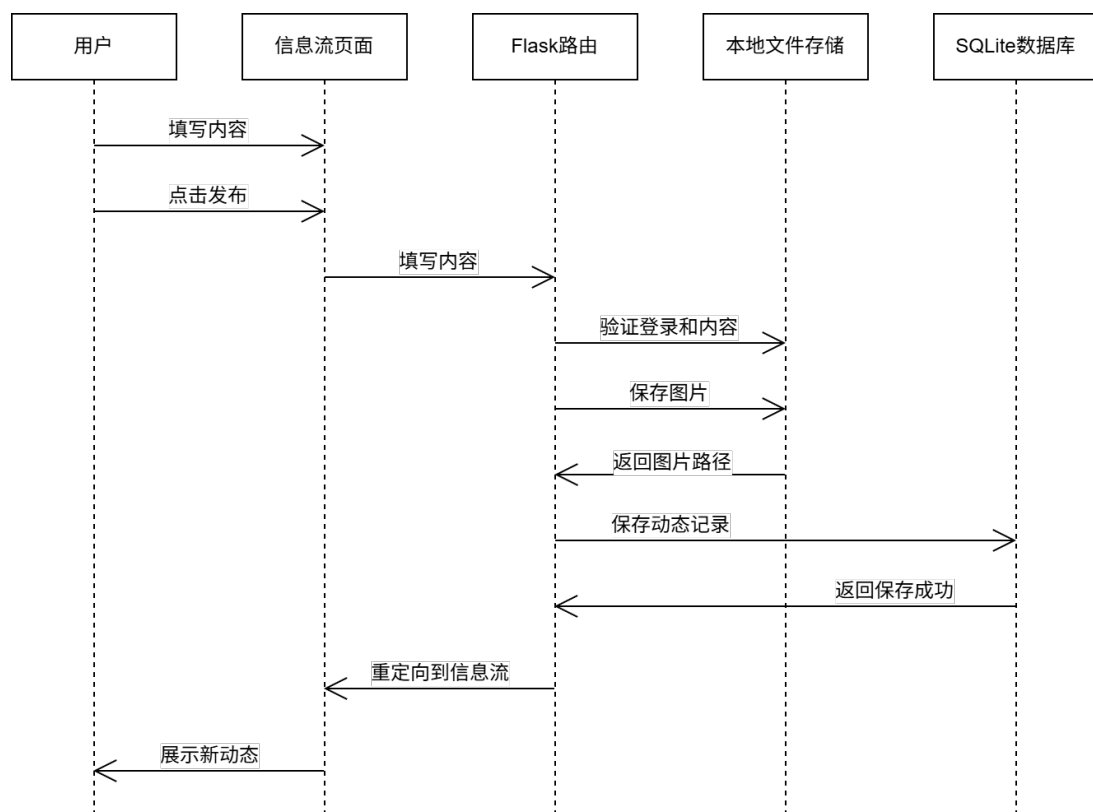


图 3: 发布动态时序图

发布动态流程说明如下：

用户在信息流页面输入动态文字内容，并可选择上传一张图片。点击发布按钮后，页面通过 POST 请求将动态内容提交至 Flask 后端。后端首先检查用户是否已经登录，并验证动态内容是否有效。若用户上传图片，系统将图片保存至本地上传目录，并记录

图片路径。随后系统将动态作者、文字内容、媒体路径、媒体类型和发布时间写入 posts 表。保存成功后，系统返回信息流页面，用户可以看到新发布的动态。

5 复杂功能算法设计

本系统的复杂功能选取“发现页推荐”进行设计。课堂演示版本采用规则式推荐算法，用于推荐未关注用户和热门公开动态。

5.1 算法流程图

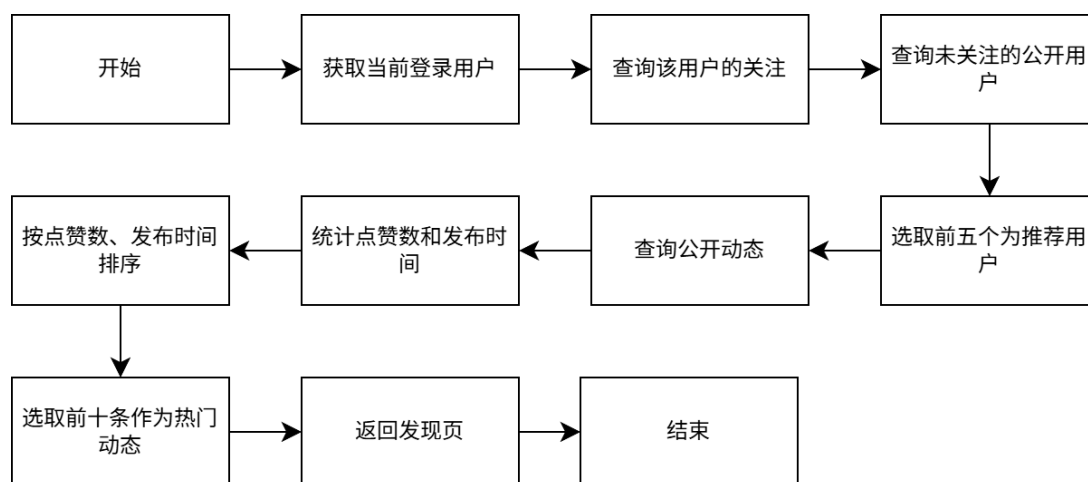


图 4: 发现页推荐算法流程图

5.2 推荐算法伪代码

输入：当前用户 `current_user`

输出：推荐用户列表 `recommended_users`，热门动态列表 `hot_posts`

1. 查询 `current_user` 已关注的用户编号列表 `following_ids`;
2. 查询所有公开用户;
3. 从公开用户中排除 `current_user` 和 `following_ids`;
4. 按注册时间或用户编号倒序排序;
5. 取前 5 个用户作为 `recommended_users`;
6. 查询所有公开动态;
7. 统计每条动态的点赞数量;
8. 按点赞数量倒序、发布时间倒序排序;
9. 取前 10 条动态作为 `hot_posts`;

10. 返回 `recommended_users` 和 `hot_posts`。

该算法不使用复杂机器学习模型，主要依据关注关系、公开状态、点赞数量和发布时间进行简单排序，适合课堂演示和小规模测试数据场景。

6 面向对象类图详细设计

系统主要类包括 `User`、`Post`、`Follow`、`Like`、`Comment`、`Conversation`、`ConversationMember`、`Message`、`Notification` 和 `UserSettings`。

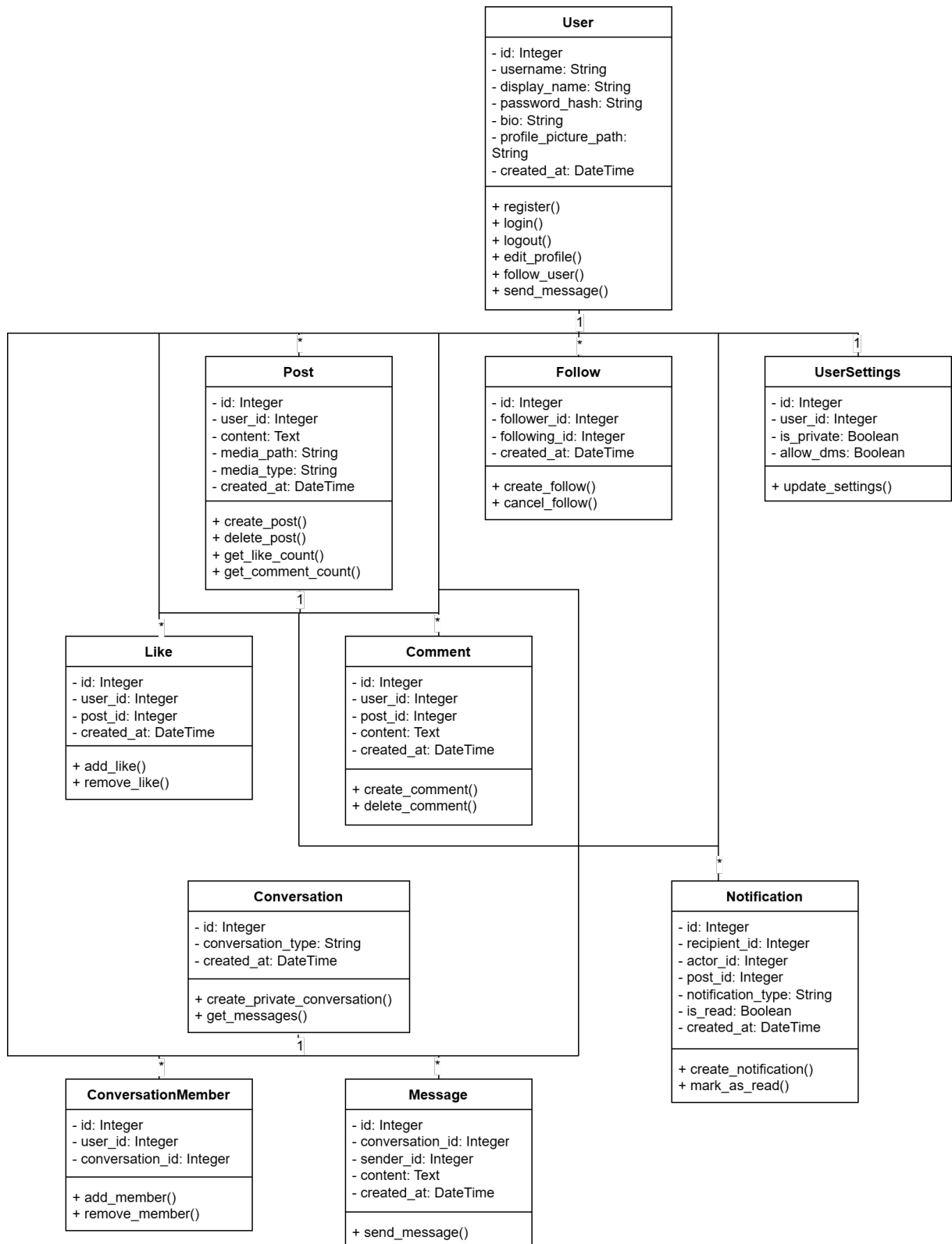


图 5: myownX 核心类图

核心类说明如下：

- User：表示系统用户，包含用户名、昵称、密码哈希、头像、简介等信息；

- Post: 表示用户发布的动态, 包含文字内容、媒体路径、媒体类型和发布时间;
- Follow: 表示用户之间的单向关注关系;
- Like: 表示用户对动态的点赞记录;
- Comment: 表示用户对动态的评论内容;
- Conversation: 表示一对一私信会话;
- ConversationMember: 表示会话成员;
- Message: 表示私信消息;
- Notification: 表示关注、点赞、评论和私信通知;
- UserSettings: 表示用户隐私设置。

主要类关系如下: 一个用户可以发布多条动态; 一个用户可以发表多条评论; 一条动态可以有多条评论和多条点赞记录; 用户之间通过 Follow 表形成关注关系; 用户通过 Conversation 和 ConversationMember 参与私信会话; Message 保存会话中的消息内容; Notification 保存互动行为产生的通知。

7 接口设计

系统采用 Flask 路由处理页面请求和表单提交。GET 请求主要用于展示页面, POST 请求主要用于提交表单和修改数据。

路由	方法	功能	登录要求
/	GET	首页或跳转信息流	否
/register	GET/POST	用户注册	否
/login	GET/POST	用户登录	否
/logout	POST	用户退出	是
/feed	GET	查看信息流	是
/posts/create	POST	发布文字或图片动态	是
/posts/<id>/delete	POST	删除自己的动态	是
/posts/<id>/like	POST	点赞或取消点赞	是
/posts/<id>/comment	POST	评论动态	是
/users/<id>	GET	查看用户主页	是
/users/<id>/follow	POST	关注或取消关注用户	是
/messages	GET	查看私信会话列表	是

路由	方法	功能	登录要求
/messages/<id>	GET/POST	查看会话或发送私信	是
/notifications	GET	查看通知列表	是
/discover	GET	查看发现页推荐内容	是
/settings	GET/POST	修改隐私设置	是

表 1: 系统接口设计表

8 数据库物理设计

系统数据库采用 SQLite，数据库文件名为 myownx.db。数据库主要保存用户、动态、关注、点赞、评论、会话、消息、通知和用户设置等结构化数据。用户头像和动态图片保存于 static/uploads/ 目录中，数据库中保存对应文件的相对路径。

数据表	说明
users	保存用户账号、昵称、密码哈希、头像、简介和注册时间
posts	保存用户发布的文字动态和图片动态
follows	保存用户之间的关注关系
likes	保存用户对动态的点赞记录
comments	保存用户对动态的评论内容
conversations	保存一对一私信会话
conversation_members	保存私信会话成员
messages	保存私信消息内容
notifications	保存关注、点赞、评论和私信通知
user_settings	保存账号公开状态和陌生人私信权限

表 2: 数据库物理表设计

主要数据库约束如下：

- users.username 设置唯一约束，防止用户名重复；
- follows(follower_id, following_id) 设置联合唯一约束，防止重复关注；
- likes(user_id, post_id) 设置联合唯一约束，防止重复点赞；
- user_settings.user_id 设置唯一约束，保证每位用户只有一条设置记录；

- posts.user_id、comments.post_id、messages.conversation_id 等字段建立外键关系；
- posts.created_at、notifications.recipient_id、messages.conversation_id 可建立普通索引，提高查询效率。

9 UI 界面设计

myownX 的界面设计参考信息流社交平台的布局策略，并结合本项目功能进行简化。系统使用 myownX 自有标识、菜单文字和测试数据。

9.1 电脑端首页设计

电脑端采用三栏布局。顶部显示 myownX 标识，左侧为功能导航栏，中间为信息流主内容区，右侧为搜索框、推荐用户和热门动态区域。用户通过左侧导航栏切换信息流、发现、通知、消息、个人主页和设置页面。

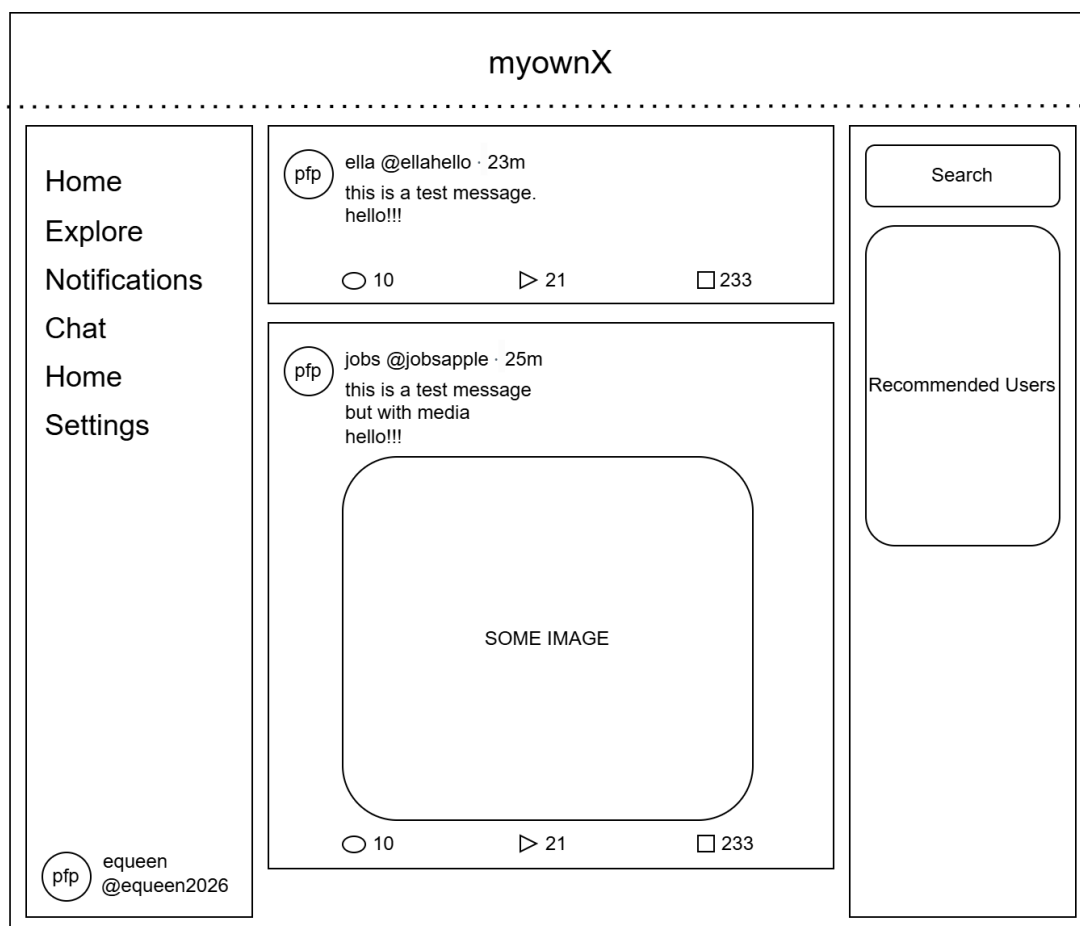


图 6: myownX 电脑端首页 UI 设计图

9.2 手机端首页设计

手机端采用顶部栏、单列信息流、底部导航栏和悬浮发布按钮结构。顶部栏左侧显示头像入口，中间显示 myownX 标识。底部导航栏提供信息流、发现、通知和消息入口，发布动态通过右下角悬浮按钮完成。



图 7: myownX 手机端首页 UI 设计图

系统页面使用 Bootstrap 5 实现响应式布局。在电脑端，页面展示左侧导航栏、中间信息流和右侧推荐栏；在手机端，左侧导航栏和右侧推荐栏隐藏，主要内容以单列方式展示，底部导航栏用于页面切换，避免页面出现明显横向滚动。

10 本地运行说明

系统开发环境如下：

- 操作系统：Windows 11；
- 开发工具：VS Code；
- 后端语言：Python 3；
- 后端框架：Flask；
- 数据库：SQLite；
- 前端技术：HTML、CSS、JavaScript、Bootstrap 5；
- 版本控制：Git。

本地运行方式如下：

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
python app.py
```

启动后，用户可在浏览器访问：

```
http://127.0.0.1:5000
```